

## BÀI THỰC HÀNH VISITOR PATTERN

- ✓ Backend: ASP.NET Core Web API
  - ✓ Frontend: React (gọi API)
  - ✓ Áp dụng **Visitor Pattern** chuẩn GoF
  - ✓ Tách **logic xử lý khỏi object structure** (đúng bản chất Visitor)
- 

## BÀI TOÁN

Hệ thống E-commerce cần xử lý **nhiều loại phần tử trong đơn hàng**:

- Product thường
- Product giảm giá
- Product digital

 Mỗi loại có cách tính:

- Giá
- Thuế
- Phí vận chuyển

 Yêu cầu:

- Không sửa class Product khi thêm logic mới
- Có thể thêm các xử lý như:
  - Tính tổng tiền
  - Tính thuế
  - Xuất invoice

 Giải pháp: **Visitor Pattern**

---

## KIẾN TRÚC HỆ THỐNG

```
Frontend (React)
    ↓
POST /api/orders/calculate
    ↓
OrderService
```

```
↓  
List<IProductElement>  
↓  
Accept(visitor)  
↓  
PriceCalculatorVisitor | TaxVisitor
```

---

## BACKEND – ASP.NET Core 8 Web API

### 1 Tạo project

```
dotnet new webapi -n VisitorPatternDemo  
cd VisitorPatternDemo
```

---

## CẤU TRÚC THƯ MỤC

```
VisitorPatternDemo  
├── Controllers  
│   └── OrdersController.cs  
├── Services  
│   └── OrderService.cs  
├── Visitors  
│   ├── IVisitor.cs  
│   ├── PriceCalculatorVisitor.cs  
│   └── TaxVisitor.cs  
├── Elements  
│   ├── IProductElement.cs  
│   ├── PhysicalProduct.cs  
│   ├── DiscountProduct.cs  
│   └── DigitalProduct.cs  
├── Models  
│   ├── OrderRequest.cs  
│   └── OrderResult.cs  
└── Program.cs
```

---

## 1. ELEMENT INTERFACE

### Elements/IProductElement.cs

```
using VisitorPatternDemo.Visitors;
```

```
namespace VisitorPatternDemo.Elements;

public interface IProductElement
{
    void Accept(IVisitor visitor);
}
```

---

## ✿ 2. CONCRETE ELEMENTS

### **PhysicalProduct.cs**

```
using VisitorPatternDemo.Visitors;

namespace VisitorPatternDemo.Elements;

public class PhysicalProduct : IProductElement
{
    public decimal Price { get; set; }

    public void Accept(IVisitor visitor)
    {
        visitor.Visit(this);
    }
}
```

---

### **DiscountProduct.cs**

```
using VisitorPatternDemo.Visitors;

namespace VisitorPatternDemo.Elements;

public class DiscountProduct : IProductElement
{
    public decimal Price { get; set; }
    public decimal Discount { get; set; }

    public void Accept(IVisitor visitor)
    {
        visitor.Visit(this);
    }
}
```

---

## DigitalProduct.cs

```
using VisitorPatternDemo.Visitors;

namespace VisitorPatternDemo.Elements;

public class DigitalProduct : IProductElement
{
    public decimal Price { get; set; }

    public void Accept(IVisitor visitor)
    {
        visitor.Visit(this);
    }
}
```

---

## ✿ 3. VISITOR INTERFACE (GOF)

### Visitors/IVisitor.cs

```
using VisitorPatternDemo.Elements;

namespace VisitorPatternDemo.Visitors;

public interface IVisitor
{
    void Visit(PhysicalProduct product);
    void Visit(DiscountProduct product);
    void Visit(DigitalProduct product);
}
```

---

## ✿ 4. CONCRETE VISITORS

### ◆ PriceCalculatorVisitor

```
using VisitorPatternDemo.Elements;

namespace VisitorPatternDemo.Visitors;
```

```

public class PriceCalculatorVisitor : IVisitor
{
    public decimal TotalPrice { get; private set; }

    public void Visit(PhysicalProduct product)
    {
        TotalPrice += product.Price + 10; // phí ship
    }

    public void Visit(DiscountProduct product)
    {
        TotalPrice += product.Price - product.Discount;
    }

    public void Visit(DigitalProduct product)
    {
        TotalPrice += product.Price; // không ship
    }
}

```

---

### ◆ TaxVisitor

```

using VisitorPatternDemo.Elements;

namespace VisitorPatternDemo.Visitors;

public class TaxVisitor : IVisitor
{
    public decimal TotalTax { get; private set; }

    public void Visit(PhysicalProduct product)
    {
        TotalTax += product.Price * 0.1m;
    }

    public void Visit(DiscountProduct product)
    {
        TotalTax += (product.Price - product.Discount) *
0.08m;
    }

    public void Visit(DigitalProduct product)

```

```
        {
            TotalTax += product.Price * 0.05m;
        }
    }
}
```

---

## ❧ 5. REQUEST & RESPONSE

### Models/OrderRequest.cs

```
namespace VisitorPatternDemo.Models;

public class OrderRequest
{
    public List<ProductDto> Products { get; set; }
}

public class ProductDto
{
    public string Type { get; set; }
    public decimal Price { get; set; }
    public decimal Discount { get; set; }
}
```

---

### Models/OrderResult.cs

```
namespace VisitorPatternDemo.Models;

public class OrderResult
{
    public decimal TotalPrice { get; set; }
    public decimal TotalTax { get; set; }
}
```

---

## ❧ 6. ORDER SERVICE

### Services/OrderService.cs

```
using VisitorPatternDemo.Elements;
using VisitorPatternDemo.Models;
using VisitorPatternDemo.Visitors;
```

```

namespace VisitorPatternDemo.Services;

public class OrderService
{
    public OrderResult Calculate(OrderRequest request)
    {
        var elements = new List<IProductElement>();

        foreach (var p in request.Products)
        {
            switch (p.Type.ToLower())
            {
                case "physical":
                    elements.Add(new PhysicalProduct {
Price = p.Price });
                    break;

                case "discount":
                    elements.Add(new DiscountProduct
                    {
                        Price = p.Price,
                        Discount = p.Discount
                    });
                    break;

                case "digital":
                    elements.Add(new DigitalProduct { Price
= p.Price });
                    break;
            }
        }

        var priceVisitor = new PriceCalculatorVisitor();
        var taxVisitor = new TaxVisitor();

        foreach (var e in elements)
        {
            e.Accept(priceVisitor);
            e.Accept(taxVisitor);
        }

        return new OrderResult
        {

```

```
        TotalPrice = priceVisitor.TotalPrice,
        TotalTax = taxVisitor.TotalTax
    };
}
}
```

---

## 7. CONTROLLER

### Controllers/OrdersController.cs

```
using Microsoft.AspNetCore.Mvc;
using VisitorPatternDemo.Models;
using VisitorPatternDemo.Services;

namespace VisitorPatternDemo.Controllers;

[ApiController]
[Route("api/orders")]
public class OrdersController : ControllerBase
{
    private readonly OrderService _service;

    public OrdersController(OrderService service)
    {
        _service = service;
    }

    [HttpPost("calculate")]
    public IActionResult Calculate(OrderRequest request)
    {
        var result = _service.Calculate(request);
        return Ok(result);
    }
}
```

---

## 8. PROGRAM.CS

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
```



```
builder.Services.AddSwaggerGen();

builder.Services.AddTransient<OrderService>();

builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowReact",
        p => p.WithOrigins("http://localhost:3000")
            .AllowAnyHeader()
            .AllowAnyMethod());
});

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseCors("AllowReact");
app.MapControllers();
app.Run();
```

---

## TEST API

```
POST /api/orders/calculate
{
  "products": [
    { "type": "physical", "price": 100 },
    { "type": "discount", "price": 200, "discount": 50 },
    { "type": "digital", "price": 150 }
  ]
}
```

---

## FRONTEND – REACT

### src/App.js

```
import React, { useState } from "react";
```

```

import axios from "axios";

function App() {
  const [result, setResult] = useState(null);

  const calculate = async () => {
    const response = await axios.post(
      "https://localhost:xxxx/api/orders/calculate",
      {
        products: [
          { type: "physical", price: 100 },
          { type: "discount", price: 200, discount: 50 },
          { type: "digital", price: 150 }
        ]
      }
    );

    setResult(response.data);
  };

  return (
    <div style={{ padding: 40 }}>
      <h2>Order Calculation</h2>

      <button onClick={calculate}>Calculate</button>

      {result && (
        <>
          <h3>Total Price: {result.totalPrice}</h3>
          <h3>Total Tax: {result.totalTax}</h3>
        </>
      )}
    </div>
  );
}

export default App;

```

---

## ▶ LUỒNG XỬ LÝ VISITOR

### 🔗 Khi gọi API

1. React gửi danh sách products
2. OrderService tạo danh sách IProductElement
3. Tạo 2 visitor:
  - PriceCalculatorVisitor
  - TaxVisitor

---

## Visitor hoạt động

```
element.Accept(visitor)
      ↓
visitor.Visit(element)
```

 Double Dispatch xảy ra:

| Element         | Visitor Method         |
|-----------------|------------------------|
| PhysicalProduct | Visit(PhysicalProduct) |
| DiscountProduct | Visit(DiscountProduct) |
| DigitalProduct  | Visit(DigitalProduct)  |

---

## ƯU ĐIỂM (ĐÚNG BẢN CHẤT VISITOR)

☒ Thêm logic mới KHÔNG sửa Product  
→ thêm Visitor mới

☒ Tách:

- Data structure (Product)
- Behavior (Visitor)

☒ Rất phù hợp:

- Tính giá
  - Tax
  - Export
  - Report
-

## NHƯỢC ĐIỂM

✕ Thêm loại Product mới → phải sửa tất cả Visitor

---

## MỞ RỘNG BÀI TẬP CHO SINH VIÊN

1. Thêm:
  - ShippingVisitor
  - InvoiceVisitor
2. Xuất:
  - PDF Invoice
3. Kết hợp:
  - Visitor + Strategy
  - Visitor + Factory